

Python

Summary Session 14



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Работа с файлами



Работа с файлами — важная часть программирования, поскольку данные часто хранятся во внешних файлах. Python позволяет создавать, читать, изменять и удалять файлы.

Экспресс-опрос

?

Почему важно уметь работать с файлами?

Почему важно уметь работать с файлами?



Хранение данных



Обмен данными



Обработка больших объёмов информации



Автоматизация работы с документами

Экспресс-опрос

?

Назовите два типа файлов

Типы файлов

Текстовые

Бинарные

Текстовые файлы



Описание

- Хранят информацию в виде обычного текста
- Используют кодировки
- Примеры расширений: .txt, .csv, .json, .html, .py

Пример

data.txt

Имя, Возраст

Анна, 25

Иван, 30

Бинарные файлы



Описание

- Хранят данные в двоичном формате (не читаемом человеком).
- Используются для хранения изображений, видео, исполняемых файлов, баз данных.
- Примеры: .jpg, .png, .mp4, .exe, .zip.

Пример

image.jpg

```
\xFF\xD8\xff\xE0\x00\x10JFIF\x00\x01\x01\x00\x00\x01\x00\x01\x00\x00
```

Экспресс-опрос

?

Для чего нужна функция `open()`



Функция open

Эта функция используется для работы с файлами в Python. Она позволяет открывать файлы для чтения и записи данных, включая текстовую и бинарную информацию. Функция возвращает объект файла.

Использование функции open



Синтаксис

```
open(file, mode="mode",  
      encoding="encoding")
```

Пояснения

- file – путь к файлу (относительный или абсолютный).
- mode (необязательный) – режим работы с файлом ("r", "w", "a", "b" и др.).
- encoding (необязательный) – кодировка текста ("utf-8", "cp1251" и др.). Используется только для текстовых файлов.

Режимы работы с файлами



При открытии файла с помощью `open()` необходимо указать режим работы.

По умолчанию файл открывается в режиме `"rt"` (чтение текста)

Экспресс-опрос

?

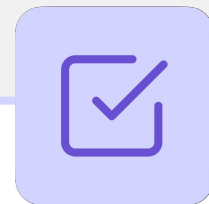
Какие режимы работы с файлами существуют?

Режимы работы с файлами



Режим	Описание
"r"	Чтение (по умолчанию). Ошибка, если файла нет.
"w"	Запись. Создаёт новый файл или перезаписывает существующий.
"a"	Добавление в конец файла. Создаёт новый, если его нет.
"x"	Создание нового файла. Ошибка, если файл уже существует.
"b"	Бинарный режим (например, "rb", "wb" для изображений).
"t"	Текстовый режим (по умолчанию).
"+"	Открытие файла как для чтения, так и для записи ("r+", "w+", "a+", "x+").

Важно



Режимы можно комбинировать, например:

- "rb" – чтение в бинарном режиме.
- "wt" – запись в текстовом режиме.
- "a+" – добавление и чтение.

Экспресс-опрос



Какой параметр используется для указания кодировки текста при чтении или записи файлов



Параметр `encoding` в функции `open()`

Этот параметр используется для указания кодировки текста при чтении или записи файлов. Он особенно важен при работе с файлами, содержащими символы, отличные от стандартных ASCII.

Почему encoding важен?



Обеспечивает правильное чтение и запись файлов



Универсальность кода



Предотвращает ошибки `UnicodeDecodeError`

Популярные кодировки



Кодировка	Описание
utf-8 (по умолчанию)	Универсальная кодировка, поддерживает все языки.
cp1251	Кодировка Windows для русского языка.
ascii	Ограничена английскими символами (0-127), устаревшая.
utf-16	Двухбайтовая кодировка, встречается в Windows-файлах.
utf-32	Четырёхбайтовая кодировка, занимает больше места.
iso-8859-1 (Latin-1)	Кодировка для западноевропейских языков.

Особенности encoding



Если encoding не указан, Python использует кодировку по умолчанию, которая зависит от операционной системы.



Используется только для текстовых файлов (при "rb" и "wb" encoding не нужен).



Неверная кодировка может вызвать UnicodeDecodeError

Экспресс-опрос

?

Какая функция используется для закрытия файла после его открытия с помощью `open()`



Функция `close()`

Эта функция используется для закрытия файла после его открытия с помощью `open()`. Закрытие файла необходимо для освобождения ресурсов и предотвращения возможных ошибок при повторном доступе к нему

Пример использования close()

```
# Можно указать просто "r", а не mode="r"
file = open("example.txt", "r", encoding="utf-8") # Открываем файл для
чтения
content = file.read() # Читаем содержимое файла
print(content)
file.close() # Закрываем файл вручную
```

Экспресс-опрос

?

Зачем закрывать файл?

Зачем закрывать файл?



Освобождение системных ресурсов

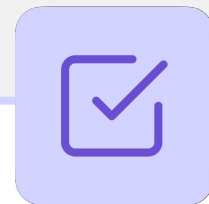


Гарантированное сохранение данных



Предотвращение ошибок доступа

Важно



Для работы с файлами используется встроенная функция `open()`, после чего файл необходимо вручную закрыть с помощью метода `close()`. Однако, если забыть закрыть файл, это может привести к утечке ресурсов.

Экспресс-опрос

?

Для чего используется контекстный менеджер with?



with

Контекстный менеджер `with` используется для безопасной работы с файлами, автоматически закрывая файл после выхода из блока. Это предпочтительный способ работы с файлами в Python

Зачем использовать `with open()`?



Автоматическое закрытие файла



Код становится чище

with open() as



Синтаксис

```
with open(file, mode="mode",
encoding="encoding") as variable:
    # Операции с файлом
```

Пояснение

- with — ключевое слово для работы с контекстными менеджерами.
- open — функция для открытия файла, возвращающая объект для работы с ним.
- as — ключевое слово для присваивания объекта файла переменной.
- variable — переменная, ссылающаяся на объект файла, через которую можно работать с ним в блоке with.

Пример использования with open()

```
with open("example.txt", "r", encoding="utf-8") as file:  
    content = file.read()  
    print(content)
```

Особенности with open()



Файл открывается только внутри блока with.



После выхода из блока файл **автоматически закрывается**.

Чтение из файла



Python предоставляет несколько способов чтения данных из файла. Когда файл открывается, указатель устанавливается в начало файла, и по мере чтения он смещается вперёд

Экспресс-опрос

?

Что такое указатель файла?



Указатель файла

Это позиция, с которой выполняется следующее чтение или запись

Экспресс-опрос

?

В чем отличие методов `read()`, `read(n)`, `readline()`, `readlines()`



Метод read()

Этот метод позволяет считать весь текст из файла в одну строку

Чтение всего содержимого файла



Пример

```
with open("example.txt", "r",  
encoding="utf-8") as file:  
    content = file.read()  
    print(type(content))  
    print(content)
```

Особенности

- Читает весь файл целиком
- Если файл большой, может занять много памяти
- Указатель после чтения перемещается в конец файла
- Переносы строк указываются как `\n`



Метод read(n)

Этот метод позволяет считать указанное количество символов

Чтение первых n символов



Пример

```
with open("example.txt", "r",
encoding="utf-8") as file:
    content = file.read(10) # Читаем
    первые 10 символов
    print(content)
```

Особенности

- Позволяет контролировать объём загружаемых данных
- Полезно при чтении больших файлов по частям



Метод readline()

Этот метод читает одну строку за вызов

Чтение построчно



Пример

```
with open("example.txt", "r",
encoding="utf-8") as file:
    print(file.readline())
    print(file.readline())
    print(file.readline())
```

Особенности

- Читает одну строку из файла
- Каждый вызов `readline()` перемещает указатель дальше
- Если в файле несколько строк, можно вызывать `readline()` повторно



Метод `readlines()`

Этот метод загружает все строки файла в список

Чтение всех строк



Пример

```
with open("example.txt", "r",
encoding="utf-8") as file:
    lines = file.readlines()
    print(type(lines))
    print(lines)
```

Особенности

- Возвращает список, где каждая строка – отдельный элемент
- Учитывает символ \n в конце строк

Чтение файла в цикле



Файл является **итерируемым объектом**, поэтому можно перебирать его содержимое **построчно в цикле** без необходимости загружать весь файл в память.

Чтение файла в цикле



Пример

```
with open("example.txt", "r",  
encoding="utf-8") as file:  
    for line in file:  
        print(line, end="")
```

Особенности

- Читает файл по одной строке, не загружая всё в память
- Подходит для обработки больших файлов

Экспресс-опрос

?

Какие методы используются для записи в файл?



Метод write()

Этот метод записывает строку в файл

Запись строки в файл: write()



Пример

```
with open("users.txt", "w",  
encoding="utf-8") as file:  
    file.write("ID: 201 | Name: John |  
Age: 25 | Status: Active\n")
```

Особенности

- Создаёт файл, если его нет
- Удаляет старые данные в файле
- write() не добавляет символ \n автоматически



Метод writelines()

Этот метод записывает список строк

Запись строки в файл: write()



Пример

```
data = [
    "ID: 202 | Name: Alice | Age: 30 |
    Status: Inactive\n",
    "ID: 203 | Name: Bob | Age: 27 |
    Status: Active\n"
]
```

```
with open("users.txt", "w",
encoding="utf-8") as file:
    file.writelines(data)
```

Особенности

- Принимает список строк и записывает их подряд
- \n нужно добавлять вручную, иначе строки сольются

Экспресс-опрос

?

Что можно делать в режимах "a", "r+", "x"?



Режим "a"

Этот режим позволяет сохранять существующее содержимое файла и дописывать данные в конец

Режим "a" – добавление в файл



Пример

```
with open("users.txt", "a",  
encoding="utf-8") as file:  
    file.write("Дополнительная  
строка\n")
```

Особенности

- Создаёт файл, если его нет
- Записывает данные в конец файла, а не перезаписывает его



Режим "r+"

Этот режим позволяет и читать, и записывать данные в файл без удаления содержимого

Режим "r+" – чтение и запись



Пример

```
with open("users.txt", "r+",
encoding="utf-8") as file:
    content = file.read() # Читаем
текущие данные
    file.write("\nДобавленный текст")
# Записываем новые данные
```

Особенности

- Файл должен существовать, иначе возникнет ошибка
- Чтение начинается с начала файла
- Запись начинается с текущей позиции указателя, что может привести к частичной перезаписи данных



Режим "x"

Этот режим используется, если нужно гарантированно создать новый файл и избежать перезаписи существующих данных

Режим "x" – создание нового файла



Пример

```
try:
    with open("new_file.txt", "x",
encoding="utf-8") as file:
        file.write("Этот файл создан в
режиме 'x'.\n")
except FileExistsError:
    print("Файл уже существует.")
```

Особенности

- Если файл уже существует, возникает `FileExistsError`
- Полезно, если важно не перезаписывать существующие данные

Экспресс-опрос

?

Позволяет ли Python открыть несколько файлов одновременно?

Контекстный менеджер с несколькими файлами



Можно открывать несколько файлов одновременно

Использование контекстного менеджера с несколькими файлами



Пример

```
with (open("example.txt", "r",  
encoding="utf-8") as infile,  
      open("output.txt", "w",  
encoding="utf-8") as outfile):  
    for line in infile:  
        outfile.write(line.upper()) #  
Записываем в верхнем регистре
```

Особенности

- Открывает сразу два файла.
- Читает из input.txt и записывает изменённый текст в output.txt

Экспресс-опрос

?

Что вы помните о формате JSON?



JSON (JavaScript Object Notation)

Это текстовый формат для хранения и передачи данных. Он представляет данные в удобном для чтения виде и используется для обмена информацией между системами

Пример JSON-объекта

```
{  
  "name": "Alice",  
  "age": 25,  
  "is_student": false,  
  "courses": ["Math", "Physics"]  
}
```

Особенности JSON

-  Структура представляет собой объекты и массивы
-  Поддерживает числа, строки, булевы значения, массивы и объекты
-  Все строки в JSON записываются только в двойных кавычках
-  Данные хранятся в текстовом виде

JSON применяется для передачи или хранения структурированных данных



API (обмен данными между системами)



Базы данных



Конфигурационные файлы



Фронтенд и бэкенд



Модуль json

Этот модуль позволяет сериализовать и десериализовать данные в формате JSON

Экспресс-опрос



Что такое сериализация?



Сериализация

Это процесс преобразования данных в формат JSON, чтобы их можно было хранить или передавать

Функции для сериализации JSON

```
json.dumps(obj)
```

```
json.dump(obj, file)
```





Функция `json.dumps`

Эта функция преобразует Python-объект в JSON-строку, чтобы передать или сохранить данные в текстовом формате

Пример использования json.dumps

```
import json

data = {'name': 'Alice', 'age': 25, 'is_student': False}

json_string = json.dumps(data) # Преобразование в JSON-строку
print(type(json_string))
print(json_string)
```



Функция `json.dump`

Эта функция записывает Python-объект в файл в формате JSON

Пример использования json.dump

```
import json

data = {"name": "Alice", "age": 25, "is_student": False}

with open("data.json", "w") as file:
    json.dump(data, file) # Запись JSON в файл
```

Применение



```
json.dumps(obj)
```

Преобразование объекта Python в JSON-строку для передачи по сети или хранения в базе данных

```
json.dump(obj, file)
```

Запись объекта Python в JSON-файл, например для сохранения конфигураций

Экспресс-опрос

?

Что такое десериализация?



Десериализация

Это процесс обратного преобразования JSON в объект Python, чтобы с ним можно было работать в программе

Функции для десериализации JSON



```
json.loads(json_string)
```

```
json.load(file)
```



Функция `json.loads`

Эта функция преобразует JSON-строку в объект Python, чтобы с ним можно было работать в коде

Пример использования json.loads

```
import json

json_object = '{"name": "Alice", "age": 25, "is_student": false}'
json_objects = '[{"name": "Alice", "age": 25, "is_student": false}, {"name": "Bob", "age": 20, "is_student": true}]'

data_dict = json.loads(json_object)    # Преобразование JSON-строки в
Python-объект
print(type(data_dict))
print(data_dict)

data_list = json.loads(json_objects)    # Преобразование JSON-строки в
Python-объект
print(type(data_list))
print(data_list)
```



Функция `json.load`

Эта функция загружает данные из JSON-файла в Python-объект

Пример использования json.load

```
import json

with open("data.json", "r") as file:
    data = json.load(file) # Загрузка JSON из файла

print(type(data))
print(data)
```

Применение



```
json.loads()
```

Если JSON приходит **в виде строки**, например, из API

```
json.dump(obj, file)
```

Если JSON **хранится в файле** и его нужно загрузить в Python

Экспресс-опрос



Какие отличия типов Python и JSON вы помните?

Сравнение типов Python и JSON

Тип в Python	Тип в JSON	Пример Python	Пример JSON
dict	object	<code>{"name": "Alice"}</code>	<code>{"name": "Alice"}</code>
list	array	<code>["apple", "banana"]</code>	<code>["apple", "banana"]</code>
tuple	array	<code>("apple", "banana")</code>	<code>["apple", "banana"]</code>
str	string	<code>"Hello"</code>	<code>"Hello"</code>
int	number	<code>42</code>	<code>42</code>
float	number	<code>3.14</code>	<code>3.14</code>
bool	boolean	<code>True, False</code>	<code>true, false</code>
NoneType	null	<code>None</code>	<code>null</code>

Особенности



Кортежи (tuple) преобразуются в массив (array), set и frozenset не поддерживаются в JSON



Булевы значения (True, False) в JSON записываются **с маленькой буквы** (true, false)



Отсутствие значения (None) в JSON записывается как null

Пример: запись в файл объекта, содержащего все типы данных

```
import json

data = {
    "dict_example": {"key": "value"},
    "list_example": ["apple", "banana"],
    "tuple_example": ("apple", "banana"),
    "string_example": "Hello",
    "int_example": 42,
    "float_example": 3.14,
    "bool_example_true": True,
    "bool_example_false": False,
    "none_example": None
}

# Запись в файл data.json
with open("data.json", "w", encoding="utf-8") as file:
    json.dump(data, file)
```

Пример: чтение обратно в Python

```
import json

# Чтение данных обратно в Python
with open("data.json", "r", encoding="utf-8") as file:
    loaded_data = json.load(file)

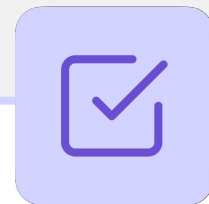
# Вывод загруженных данных
print(type(loaded_data))
print(loaded_data)
```

Экспресс-опрос

?

Какой JSON по умолчанию создают `json.dumps()` и `json.dump()`?

Важно



По умолчанию `json.dumps()` и `json.dump()` создают **однострочный JSON**, что делает его менее читабельным.

Параметры для форматирования JSON

`indent`

`ensure_ascii`

`sort_keys`

Экспресс-опрос

?

Для чего используется параметр `indent`?



Параметр indent

Этот параметр используется для добавления отступов в JSON, чтобы сделать его более читаемым

Пример использования indent

```
import json

data = {"name": "Alice", "age": 25, "is_student": False,
        "courses": {"math": "A", "physics": "B" }}

json_string = json.dumps(data) # Без отступа
print(json_string)

json_string = json.dumps(data, indent=4) # С отступом
print(json_string)
```

Экспресс-опрос

?

Для чего используется параметр `ensure_ascii`?



Параметр `ensure_ascii`

Этот параметр определяет, как JSON будет хранить юникод-символы

Пример использования ensure_ascii

```
import json

data = {"город": "Берлин", "страна": "Германия"}

json_string = json.dumps(data) # По умолчанию (ensure_ascii=True)
print(json_string)

json_string = json.dumps(data, ensure_ascii=False) # Отключаем ASCII-
кодировку
print(json_string)
```

Экспресс-опрос

?

Для чего используется параметр `sort_keys`?



Параметр `sort_keys`

Этот параметр используется для сортировки ключей в JSON-объекте по алфавиту

Пример использования sort_keys

```
import json

data = {"name": "Alice", "age": 25, "is_student": False, "city": "London"}

json_string = json.dumps(data, indent=4) # Без сортировки ключей
print(json_string)

json_string = json.dumps(data, indent=4, sort_keys=True) # Сортировка
ключей
print(json_string)
```

Экспресс-опрос

?

В каких случаях возникает JSONDecodeError?



JSONDecodeError

Эта ошибка возникает, если строка JSON имеет неверный формат и не может быть разобрана с помощью `json.loads()` или `json.load()`. Она указывает, что JSON-данные повреждены, содержат синтаксические ошибки или не соответствуют ожиданиям

Пример возникновения JSONDecodeError

```
import json

invalid_json = '{"name": "Alice", "age": 25, "is_student": false,' #
Ошибка: нет закрывающей скобки

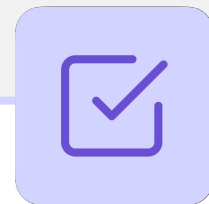
data = json.loads(invalid_json) # Загрузка некорректного JSON
```

Экспресс-опрос

?

Что помогает избежать ошибки при загрузке JSON?

Важно



Используйте `try-except**` при загрузке JSON, чтобы избежать ошибки

Пример использования try-except**

```
import json

invalid_json = '{"name": "Alice", "age": 25, "is_student": false,' #
Ошибка: нет закрывающей скобки

try:
    data = json.loads(invalid_json) # Попытка загрузки некорректного
JSON
except json.JSONDecodeError as e:
    print(f"Ошибка декодирования JSON: {e}")
```

Экспресс-опрос



Какие причины возникновения `JSONDecodeError` вы помните?

Причины JSONDecodeError

Причина

Пропущенные кавычки или
запятые

Использование одинарных
кавычек вместо двойных

Неполные или повреждённые
данные

Пример

```
{"name": "Alice", "age": 25, "is_student": false,} #  
Лишняя запятая
```

```
{'name': 'Alice'} # Неверный формат
```

```
{"name": "Alice", "age": 25 # Нет закрывающей скобки
```

Экспресс-опрос

?

Какой модуль предоставляет инструменты для работы с датами, временем и их форматированием?



Модуль datetime

Этот модуль предоставляет инструменты для работы с датами, временем и их форматированием.

Он позволяет получать текущее время, вычислять разницу между датами, конвертировать форматы и работать с часовыми поясами.



Метод `datetime.now()`

Этот метод модуля `datetime` используется для получения текущей даты и времени

Пример получения текущей даты и времени

```
from datetime import datetime

now = datetime.now() # Получаем текущую дату и время
print(type(now))    # Объект datetime
print(now)
```

Особенности



Возвращает объект `datetime` с текущей датой и временем



Позволяет извлекать отдельные компоненты даты

Экспресс-опрос

?

Какие примеры применения `datetime.now()` вы можете привести?

Зачем это нужно



Фиксация времени событий



Отметка времени при выполнении операций



Создание временных меток

Пример получения компонентов даты

```
from datetime import datetime

now = datetime.now()

print("Год:", now.year)
print("Месяц:", now.month)
print("День:", now.day)
print("Часы:", now.hour)
print("Минуты:", now.minute)
print("Секунды:", now.second)
```

Экспресс-опрос

?

Что делает метод `strftime()`?



Метод `strftime()`

Этот метод используется для преобразования даты в строку в нужном формате

Пример

```
from datetime import datetime

now = datetime.now()
# Преобразование в строку
print(str(now))
# Преобразование в строку указанного формата
formatted_date = now.strftime("%d.%m.%Y %H:%M:%S")
print(type(formatted_date))
print(formatted_date)
```

Популярные коды форматов strftime

Код	Описание	Пример
%d	День (01-31)	28
%m	Месяц (01-12)	02
%Y	Год (4 цифры)	2025
%y	Год (2 цифры)	25
%H	Часы (00-23)	14
%M	Минуты (00-59)	30
%S	Секунды (00-59)	15
%A	Полное название дня	Friday
%B	Полное название месяца	February

Примеры форматирования

```
from datetime import datetime

now = datetime.now()
print(now.strftime("%Y-%m-%d"))      # 2025-02-28 (ISO формат)
print(now.strftime("%d/%m/%Y"))      # 28/02/2025 (европейский формат)
print(now.strftime("%I:%M %p"))      # 02:30 PM (12-часовой формат)
print(now.strftime("%A, %B %d, %Y")) # Friday, February 28, 2025
```

Экспресс-опрос

?

Какой метод используется для преобразования строки в объект datetime?



Метод `strptime()`

Этот метод используется для преобразования строки в объект `datetime`.

Это необходимо, когда дата хранится в виде текста (например, в файле) и её нужно использовать для вычислений или фильтрации

Пример

```
from datetime import datetime

date_string = "28|02|2025 14-30-15" # Дата в виде строки
date_obj = datetime.strptime(date_string, "%d|%m|%Y %H-%M-%S") #
# Указываем форматы и те же разделители

print(type(date_obj))
print(date_obj)
```

Сравнение дат



Python позволяет **сравнивать даты** так же, как числа, используя операторы сравнения (`>`, `<`, `==`, `!=`, `>=`, `<=`).
Объекты `datetime` можно сравнивать напрямую, так как они содержат **и дату, и время**.

Пример: разбор даты из строки и сравнение с текущей датой**

```
from datetime import datetime

now = datetime.now()
deadline = datetime.strptime("01.12.2025", "%d.%m.%Y")

if now > deadline:
    print("Срок истёк!")
else:
    print("До дедлайна ещё есть время.")
```

Разница между датами



Python позволяет **вычислять разницу между датами** с помощью **вычитания объектов datetime**, которое возвращает объект `timedelta`.

Пример: Разница между датами

```

from datetime import datetime

date1 = datetime(2025, 2, 28)
date2 = datetime(2025, 3, 5)

difference = date2 - date1 # Разница между датами
print(type(difference))
print(difference)
print(difference.days)
  
```

Пример: Разница с учётом времени

```
from datetime import datetime

dt1 = datetime(2025, 2, 28, 14, 30)
dt2 = datetime(2025, 3, 2, 10, 0)

difference = dt2 - dt1
print(difference)
print(difference.total_seconds())
```

Экспресс-опрос

?

Какой объект можно использовать для изменения дат, добавляя или вычитая временные интервалы?

Полезно знать



Объект `timedelta` можно использовать для **изменения дат**, добавляя или вычитая временные интервалы.

Пример: Добавление и вычитание времени

```
from datetime import datetime, timedelta

# Дата начала задачи
start_date = datetime(2025, 2, 28)

# Дедлайн через 2 недели
deadline = start_date + timedelta(weeks=2)
print("Дедлайн:", deadline.strftime("%d.%m.%Y"))

# Проверка, прошёл ли дедлайн
today = datetime(2025, 3, 15) # Текущая дата

if deadline > today:
    print("Дедлайн пропущен!")
else:
    print("Ещё есть время для выполнения задачи.")
```



ВОПРОСЫ





ЗАДАНИЕ





Выберите верный вариант ответа

Что делает "a" в функции open ()? Перечислите все варианты.

- a. Перезаписывает файл
- b. Добавляет в конец файла
- c. Читает файл
- d. Создаёт файл, если его нет



Выберите верный вариант ответа

Что делает "a" в функции open ()? Перечислите все варианты.

- a. Перезаписывает файл
- b. Добавляет в конец файла
- c. Читает файл
- d. Создаёт файл, если его нет



Выберите верный вариант ответа

Какой метод закрывает файл после работы?

- a. `terminate()`
- b. `close()`
- c. `stop()`
- d. `exit()`



Выберите верный вариант ответа

Какой метод закрывает файл после работы?

- a. `terminate()`
- b. `close()`
- c. `stop()`
- d. `exit()`



Выберите верный вариант ответа

Что произойдёт, если не закрыть файл после `open()`?

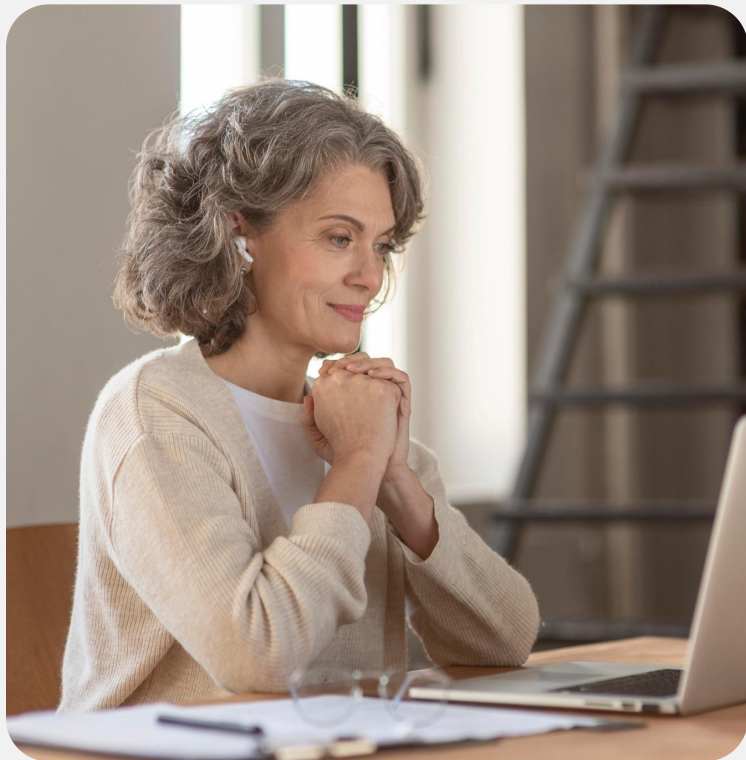
- a. Файл останется открытым в памяти
- b. Ничего страшного, Python сам закроет
- c. Возникнет `FileNotFoundException`
- d. Запись в файл может не сохраниться



Выберите верный вариант ответа

Что произойдёт, если не закрыть файл после `open()`?

- a. Файл останется открытым в памяти
- b. Ничего страшного, Python сам закроет
- c. Возникнет `FileNotFoundException`
- d. Запись в файл может не сохраниться



Выберите верный вариант ответа

Какой режим используется для добавления
данных в конец файла?

- a. "r"
- b. "w"
- c. "a"
- d. "x"



Выберите верный вариант ответа

Какой режим используется для добавления
данных в конец файла?

- a. "r"
- b. "w"
- c. "a"
- d. "x"



Выберите верный вариант ответа

Что произойдёт, если открыть файл в режиме "x", а он уже существует?

- a. Файл будет перезаписан
- b. Возникнет `FileExistsError`
- c. Файл откроется для чтения
- d. Файл откроется для записи



Выберите верный вариант ответа

Что произойдёт, если открыть файл в режиме "x", а он уже существует?

- a. Файл будет перезаписан
- b. Возникнет `FileExistsError`**
- c. Файл откроется для чтения
- d. Файл откроется для записи



Какой результат даст следующий код?

```
import json
```

```
data = {"имя": "Алиса", "город": "Минск"}  
json_str = json.dumps(data)  
print(json_str)
```



Какой результат даст следующий код?

```
import json
```

```
data = {"имя": "Алиса", "город": "Минск"}  
json_str = json.dumps(data)  
print(json_str)
```

Ответ: Строка будет содержать символы в кодировке \uXXXX



Сопоставь Python-типу его соответствие в JSON:

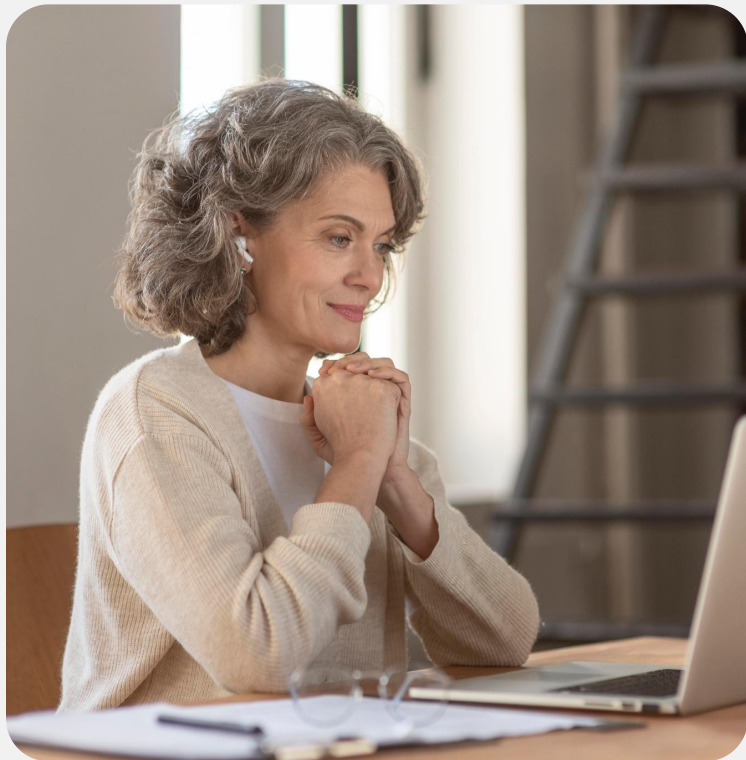
- | | |
|-------------|------------|
| 1. NoneType | a. array |
| 2. bool | b. null |
| 3. tuple | c. object |
| 4. dict | d. string |
| 5. list | e. boolean |
| 6. str | |



Сопоставь Python-типу его соответствие в JSON:

- | | |
|-------------|------------|
| 1. NoneType | a. array |
| 2. bool | b. null |
| 3. tuple | c. object |
| 4. dict | d. string |
| 5. list | e. boolean |
| 6. str | |

Ответ: 1-b, 2-e, 3-a, 4-c, 5-a, 6-d



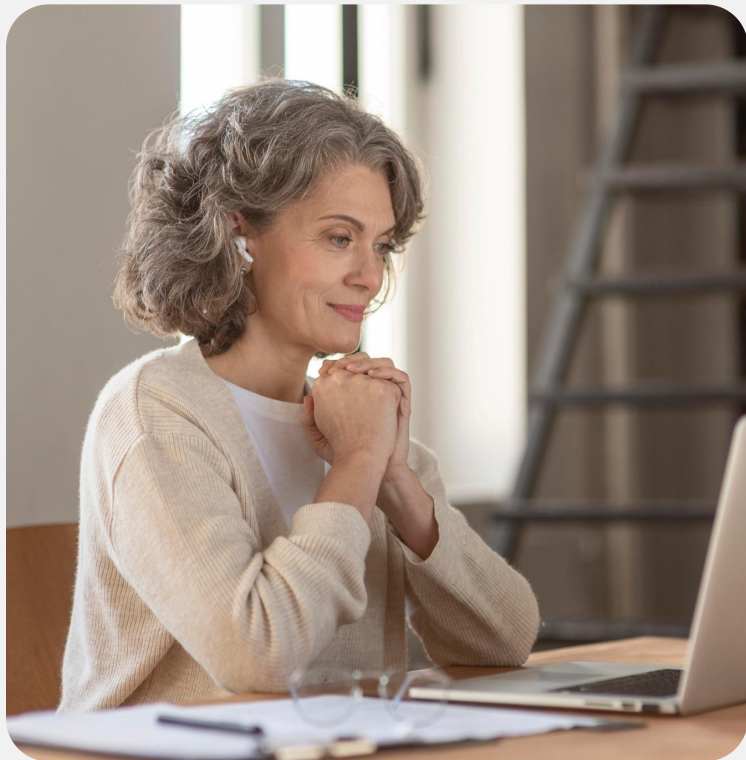
Выберите верный вариант ответа

Что выведет следующий код?

```
from datetime import datetime, timedelta
```

```
start = datetime(2025, 1, 30)
delta = timedelta(days=10)
new_date = start + delta
print(new_date.strftime("%d-%m-%Y"))
```

- a. 08-02-2025
- b. 09-02-2025
- c. 10-02-2025
- d. 11-02-2025



Выберите верный вариант ответа

Что выведет следующий код?

```
from datetime import datetime, timedelta
```

```
start = datetime(2025, 1, 30)
```

```
delta = timedelta(days=10)
```

```
new_date = start + delta
```

```
print(new_date.strftime("%d-%m-%Y"))
```

a. 08-02-2025

b. 09-02-2025

c. 10-02-2025

d. 11-02-2025



Сопоставь формат с результатом:

- | | |
|---------------|-----------------------|
| 1. "%Y.%m.%d" | a. Friday.28.February |
| 2. "%A.%d.%B" | b. 2025.02.28 |
| 3. "%B.%d.%Y" | c. 28.02.2025 |
| 4. "%d.%m.%Y" | d. February.28.2025 |

Ответ: 1-b, 2-a, 3-d, 4-c



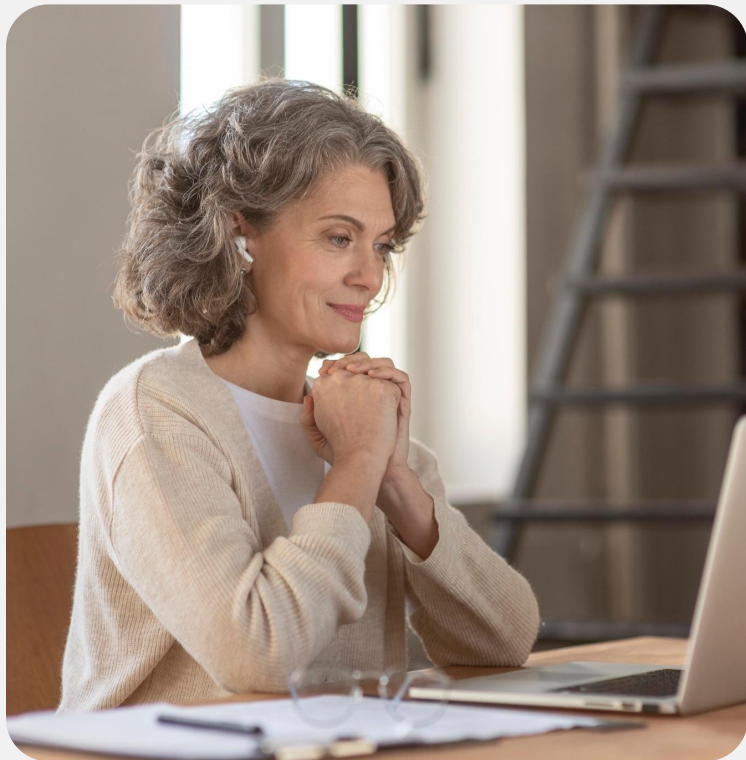
Выберите верный вариант ответа

Что выведет следующий код?

```
from datetime import datetime
```

```
dt = datetime(2025, 2, 28, 14, 30, 15)  
print(dt.strftime("%H:%M"))
```

- a. 2:30
- b. 14:30
- c. 14.30
- d. 02:30



Выберите верный вариант ответа

Что выведет следующий код?

```
from datetime import datetime
```

```
dt = datetime(2025, 2, 28, 14, 30, 15)
print(dt.strftime("%H:%M"))
```

- a. 2:30
- b. 14:30
- c. 14.30
- d. 02:30



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА



Задание 1. Подсчёт среднего балла учеников

Напишите программу, которая запрашивает у пользователя имя файла с оценками учеников, подсчитывает средний балл каждого ученика и выводит результат на экран.

Если указанный файл не существует, программа должна вывести ошибку.

Используйте файл grades.txt.

Пример вывода:

Alice: 86.0

Bob: 87.0

Charlie: 84.8

2. Поиск просроченных задач

Реализовать программу, которая должна:

- Прочитать задачи из файла `tasks_with_deadline.json`.
 - Каждая задача содержит **название**, **срок выполнения** и **статус** (done или pending).
- Запросить у пользователя дату в формате dd-mm-yyyy или использовать текущую.
- Найти все задачи, которые:
 - **не выполнены** (pending)
 - и **срок которых истёк на указанную дату**
- Вывести список просроченных задач на экран.

2. Поиск просроченных задач

Данные:

Файл tasks_with_deadline.json:

```
[
  {"title": "Audit finances", "deadline": "23-12-2025", "status": "done"},
  {"title": "Schedule interviews", "deadline": "10-06-2025", "status": "done"},
  {"title": "Update CRM", "deadline": "08-02-2025", "status": "done"},
  {"title": "Plan event", "deadline": "15-09-2025", "status": "pending"},
  {"title": "Team meeting", "deadline": "15-04-2025", "status": "pending"},
  {"title": "Plan event", "deadline": "23-05-2025", "status": "pending"},
  {"title": "Backup server", "deadline": "19-03-2025", "status": "pending"},
  {"title": "Host webinar", "deadline": "20-02-2025", "status": "done"},
  {"title": "Sign contract", "deadline": "28-07-2025", "status": "pending"},
  ...
]
```

2. Поиск просроченных задач

Пример вывода (если введена дата 12-03-2025):

Введите дату проверки (dd-mm-yyyy) или нажмите Enter для текущей: 03-03-2025

Просроченные задачи:

- Post job ad (17-02-2025)
- Sign contract (08-01-2025)
- Team meeting (06-01-2025)
- Update website (22-02-2025)
- Prepare slides (12-02-2025)
- Review performance (04-01-2025)
- Refactor code (31-01-2025)



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Фильтрация по ключевому слову

Напишите программу, которая ищет в файле все строки, содержащие указанное пользователем слово, и сохраняет их в новый файл.

- Имя нового файла формируется как `<keyword>_<original_filename>`.
- Если файл не существует, программа должна вывести ошибку.
- Если совпадения не найдены, новый файл не создаётся.

Используйте файл `system_log.txt`.

Пример ввода:

Введите имя файла для поиска: `system_log.txt`
Введите ключевое слово: `error`

Пример вывода:

Строки, содержащие `'error'`,
сохранены в `error_system_log.txt`.

Домашнее задание

Поиск и удаление файлов с указанным расширением

2. Поиск и удаление дубликатов

Напишите программу, которая удаляет дублирующиеся строки из файла и сохраняет результат в новый файл.

- Имя нового файла формируется как `unique_<original_filename>`.
- Если файл не существует, программа должна вывести ошибку.
- Исходный порядок строк должен сохраниться.
- Если в файле нет дубликатов, создаётся точная копия файла.

Используйте файл `movies_to_watch.txt`.

Пример ввода:

Введите имя файла: `movies_to_watch.txt`

Пример вывода:

Дубликаты удалены. Уникальные строки сохранены в `unique_movies_to_watch.txt`.



ВОПРОСЫ



Домашнее задание

Анализ курсов студентов

Реализуйте программу, которая должна:

1. Прочитать файл `student_courses.json`, содержащий:
 - a. Имя,
 - b. дату рождения (`birth_date`) в формате дд.мм.гггг,
 - c. дату поступления (`enrollment_date`) в том же формате,
 - d. список курсов.
2. Вычислить:
 - a. Общее количество студентов.
 - b. Средний возраст на момент поступления.
 - c. Количество студентов на каждом курсе.
3. Сохранить отчёт в JSON-файл `student_courses_report.json`.

Домашнее задание

Анализ курсов студентов

Данные:

```
[
  {"name": "Diana Williams", "birth_date": "12.06.1983", "enrollment_date": "29.04.2023", "courses": ["Physics",
"Chemistry"]},
  {"name": "Tina Miller", "birth_date": "06.07.2004", "enrollment_date": "18.04.2020", "courses": ["Biology", "Business"]},
  {"name": "Kevin Miller", "birth_date": "20.12.2004", "enrollment_date": "16.12.2020", "courses": ["Linguistics", "Math",
"History"]},
  {"name": "Fiona Brown", "birth_date": "05.07.1999", "enrollment_date": "02.09.2022", "courses": ["Art", "Philosophy"]},
  {"name": "Charlie Davis", "birth_date": "17.07.1998", "enrollment_date": "17.05.2023", "courses": ["Chemistry", "Physics",
"Business"]},
  {"name": "Diana Jones", "birth_date": "24.12.1980", "enrollment_date": "26.11.2021", "courses": ["Economics",
"Linguistics"]},
  {"name": "Alice Johnson", "birth_date": "22.09.1981", "enrollment_date": "23.12.2020", "courses": ["Chemistry", "Economics",
"Math"]},
  {"name": "Ian Lopez", "birth_date": "23.11.2001", "enrollment_date": "07.05.2020", "courses": ["Philosophy", "Art",
"Physics"]},
  {"name": "Kevin Davis", "birth_date": "30.01.1997", "enrollment_date": "20.03.2021", "courses": ["Math", "Economics"]},
  ...
]
```

Домашнее задание

Анализ курсов студентов

Пример вызова:

```
filter_low_scores(70, "01-01-2025", "31-03-2025")
```



ВОПРОСЫ



Заключение

